

**M18 / LEARNER BOOK**

# Portfolio projects and production capstone

## Standalone Zero-to-Job-Ready Data Engineer 2026

50 controlled project phases

### A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

# Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| START HERE   How to use this module . . . . .                                  | 5  |
| P01-01   Read the batch-pipeline brief and define business questions . . . . . | 6  |
| P01-02   Profile every source; state grain, keys and defects . . . . .         | 8  |
| P01-03   Draw architecture and create the repository scaffold . . . . .        | 10 |
| P01-04   Preserve raw landing evidence and source/run metadata . . . . .       | 13 |
| P01-05   Build ingestion with validation and quarantine . . . . .              | 15 |
| P01-06   Add incremental state and rerun-safe identity . . . . .               | 17 |
| P01-07   Build trusted PostgreSQL/dbt-style transformations . . . . .          | 20 |
| P01-08   Add tests and source-to-target reconciliation . . . . .               | 22 |
| P01-09   Automate the repeatable run/orchestration path . . . . .              | 25 |
| P01-10   Inject failure, diagnose, repair and prove recovery . . . . .         | 28 |
| P01-11   Finish README, runbook, architecture and evidence index . . . . .     | 30 |
| P01-12   Clean-clone rerun and project defense . . . . .                       | 33 |
| P02-01   Read the analytics-engineering brief and define outputs . . . . .     | 36 |
| P02-02   Profile billing/customer sources and state grains . . . . .           | 38 |
| P02-03   Design facts, dimensions, keys and serving grain . . . . .            | 40 |
| P02-04   Create dbt project, sources and dependency graph . . . . .            | 42 |
| P02-05   Build thin staging models with explicit contracts . . . . .           | 44 |
| P02-06   Build dimensional marts without fan-out . . . . .                     | 46 |
| P02-07   Add data tests and critical quality gates . . . . .                   | 48 |
| P02-08   Implement incremental model/update behavior . . . . .                 | 51 |

|                                                                              |     |
|------------------------------------------------------------------------------|-----|
| P02-09   Preserve history with snapshot/SCD reasoning . . . . .              | 54  |
| P02-10   Add orchestration and reproducible run order . . . . .              | 56  |
| P02-11   Inject duplicate/relationship/partial-run failures . . . . .        | 59  |
| P02-12   Document lineage, runbook, limitations and evidence . . . . .       | 61  |
| P02-13   Clean-clone build and analytics-platform defense . . . . .          | 63  |
| P03-01   Read the scalable-lakehouse brief and define scale risks . . . . .  | 66  |
| P03-02   Profile event schema, grain, keys and timestamps . . . . .          | 69  |
| P03-03   Create Spark environment and explicit schemas . . . . .             | 71  |
| P03-04   Build deterministic distributed transformations . . . . .           | 74  |
| P03-05   Join reference data and validate cardinality . . . . .              | 77  |
| P03-06   Reason about partitions, shuffle and skew . . . . .                 | 79  |
| P03-07   Write Parquet with justified partition strategy . . . . .           | 82  |
| P03-08   Apply Delta-style incremental merge/idempotency . . . . .           | 85  |
| P03-09   Define Bronze/Silver/Gold lakehouse contracts . . . . .             | 88  |
| P03-10   Handle schema evolution and supplied failure cases . . . . .        | 91  |
| P03-11   Inspect plans/performance and prove equivalent results . . . . .    | 93  |
| P03-12   Map the design to Fabric/cloud deployment . . . . .                 | 96  |
| P03-13   Clean rerun, reconcile outputs and defend trade-offs . . . . .      | 98  |
| CAP-01   Read capstone requirements and write source contracts . . . . .     | 101 |
| CAP-02   Design architecture and record major ADRs . . . . .                 | 103 |
| CAP-03   Build multi-source ingestion and raw evidence . . . . .             | 105 |
| CAP-04   Implement critical quality, quarantine and reconciliation . . . . . | 107 |
| CAP-05   Build trusted warehouse/dbt serving layer . . . . .                 | 109 |

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| CAP-06   Orchestrate dependencies, retries and bounded backfills . . . . . | 111 |
| CAP-07   Use Spark/Delta where scale justifies it . . . . .                | 114 |
| CAP-08   Design Fabric deployment, access and secret boundaries . . . . .  | 117 |
| CAP-09   Add CI/CD, monitoring, alerts and runbook . . . . .               | 119 |
| CAP-10   Open incident pack, diagnose and recover safely . . . . .         | 122 |
| CAP-11   Reproduce from clean state and freeze evidence bundle . . . . .   | 124 |
| CAP-12   Final technical defense and Gate 7 readiness . . . . .            | 126 |

**START HERE**

# How to use this module

Open the matching practice folder from the course root: 17\_M18/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Portfolio rule: You are not graded for copying a tutorial. Every phase must leave your own evidence: first attempt, run/output, validation, one meaningful failure/recovery when assigned, changed-input retry and a short defense. Videos are visual support; the repository must stand on its own.

Outcome. By the end of M18 you will have three focused repositories plus one production-capstone evidence bundle that demonstrate reliable batch engineering, analytics engineering/dbt thinking, scalable Spark/lakehouse reasoning and an integrated production data-platform design.

Project route: P01 12 phases -> P02 13 -> P03 13 -> Capstone 12. Work in the matching folder under 17\_M18/practice/. Protected project reviews are not part of the normal first attempt.

A regional parcel company receives daily order CSVs plus customer/product reference data. Build a replayable batch pipeline that preserves raw evidence, quarantines invalid rows, supports incremental reruns, produces trusted serving outputs and survives a deliberate failure.

P01-01

# Read the batch-pipeline brief and define business questions

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

The business question is how much PAID order revenue is trusted after each daily batch. One output row represents one current order, not one arrival. Write that sentence before writing code. Revenue is qty multiplied by the product unit\_price; the order files do not contain a ready-made revenue column. PENDING and CANCELLED orders remain in Silver but contribute zero to paid revenue.

## Artifact and completion check

Create docs/source-contract.md. Name the input files, the order\_id key, the updated\_at version, and the status rule. A reader should be able to calculate one order by hand from your contract.

Controlled phase 1 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the supplied project brief once without coding.
2. Write the business questions, decision deadline, data consumers and non-functional constraints.

3. List what is evidence versus assumption.
4. Define the exact artifact that will prove the phase complete before creating code.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to read the batch-pipeline brief and define business questions - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-02

# Profile every source; state grain, keys and defects

## Watch first

### [1. Python for Data Analytics - profiling with pandas](#)

Optional visual - no verified bounded range. Focus on DataFrame inspection, nulls, duplicates and grouped counts. Then profile the project files, not the instructor dataset.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Count physical rows before deduplication. Day 1 has 12 rows and 11 distinct order IDs. Those numbers do not imply 11 valid orders: two records fail a relationship or quantity rule, and one valid order appears twice identically. Profile customers and products separately before joining. A repeated key in a lookup would multiply orders even if the order parser is correct.

## Artifact and completion check

Create evidence/source-profile.json. Record 12 = 9 accepted unique orders + 1 identical duplicate + 2 quarantined rows. Identify the raw records behind every category.

Controlled phase 2 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Copy source files into the working project without altering the originals.

2. Record row counts and the meaning of one row for every source.
3. Check keys, duplicates, missing values, type surprises and cross-source relationships.
4. Save a short data-quality note before you decide what to repair.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- State row grain, candidate keys, null/duplicate counts and at least three concrete defects.
- Do not clean the source before preserving the profiling evidence.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Pretend the first 20 rows represent the whole source. A duplicate or malformed key later in the file then breaks your assumed grain.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to profile every source; state grain, keys and defects - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-03

# Draw architecture and create the repository scaffold

## Watch first

### [1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

### [2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Draw acquisition, immutable raw, validation, current orders, paid summary and progress as separate responsibilities. Point rejected rows to quarantine. Put the progress write after the successful output checks. Keep source code under src, tests under tests and ignored generated data under output. A diagram is useful only if every box has a corresponding file, table or function.

## Artifact and completion check

Create docs/architecture.md. Trace one valid row and one invalid row through your drawing. Mark which existing artifact lets you rebuild after a parser bug.

Controlled phase 3 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Draw source -> raw -> validation -> transform -> serving -> consumer boundaries.

2. Mark durable writes, checkpoints/state and security boundaries.
3. Record at least two alternatives and why one is rejected.
4. Save the diagram plus an ADR so reviewers can challenge your reasoning.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Every major component must trace to a requirement/constraint.
- Record at least one rejected alternative and the evidence that would make you revisit the decision.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: git](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Reuse a run ID with changed bytes. If the earlier raw object disappears, your audit trail is not immutable.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to draw architecture and create the repository scaffold - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-04

# Preserve raw landing evidence and source/run metadata

## Watch first

### [1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Use the ingestion demonstration for acquisition and validation context; implement the project-specific quarantine, identity and recovery rules below.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Copy source bytes before parsing them. Record source name, byte count and SHA-256. A hash describes the bytes you received; `order_id` describes a business entity. They solve different problems. Reusing a run ID with identical bytes may be a harmless replay. Reusing it with changed bytes must fail instead of silently replacing the original acquisition.

## Artifact and completion check

Create `output/raw/<run-id>/_manifest.json`. Land the same file twice under one ID; then change `qty` in a COPY and try that ID again. Original bytes and manifest must remain intact after the rejected attempt.

Controlled phase 4 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Create an immutable landing path using source + run identity.

2. Write bytes first, calculate hash/size/metadata, then append manifest state.
3. Test identical replay and changed-content collision.
4. Prove the first landed bytes remain retrievable.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Raw bytes must be immutable and retrievable by run/source identity.
- A repeated run must not silently overwrite earlier evidence.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Reuse a run ID with changed bytes. If the earlier raw object disappears, your audit trail is not immutable.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to preserve raw landing evidence and source/run metadata - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-05

# Build ingestion with validation and quarantine

## Watch first

### [1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Use the ingestion demonstration for acquisition and validation context; implement the project-specific quarantine, identity and recovery rules below.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

For each raw order, check that qty is finite, whole and positive, the timestamp parses, customer and product exist, and status is one of the declared values. Preserve the raw record and a useful reason when rejecting it. Validation precedes aggregation, so a negative quantity cannot quietly reduce paid revenue. Deduplication is a separate count, not a quarantine reason.

## Artifact and completion check

Create output/quarantine/orders.jsonl. Day 1 must have 2 quarantines, 1 identical duplicate and 9 current valid orders. Inspect customer C999 and the negative-quantity row.

Controlled phase 5 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define acceptance rules before reading outputs.

2. Separate business-invalid records from infrastructure failure.
3. Write accepted and quarantine populations durably, then reconcile counts.
4. Inject one invalid case and prove the reason is clear.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Accepted + quarantined must reconcile to the processed input population.
- Invalid reasons must be actionable; infrastructure failure must not be mislabeled as bad data.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build ingestion with validation and quarantine - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-06

# Add incremental state and rerun-safe identity

## Watch first

### [1. M09 retained visual - incremental processing](#)

Optional visual - no verified bounded range

### [2. Databricks - How Delta Lake Supercharges Data Lakes](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Compare typed versions for each `order_id`. A strictly newer version replaces the current order; an identical replay does nothing; equal version with different content is a conflict. O1004 changes from PENDING to PAID in day 2. O1012 and O1013 are new orders. You therefore process three changes but increase the current order count by only two.

## Artifact and completion check

Create `output/state/orders.json`. After day 2, there are 11 current orders. Replaying day 2 changes neither that count nor the paid total. Do not advance progress after a rejected conflict.

Controlled phase 6 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define ordering/version semantics and candidate progress.
2. Apply changes to a temporary/transactional result, validate it, then commit progress.

3. Replay the same batch and then an older batch.
4. Prove business state and progress remain monotonic under the stated contract.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Identical replay leaves trusted business state unchanged.
- Stale input cannot move progress backwards; equal-version ambiguity is surfaced rather than guessed.

## Useful video / current reference

[VIDEO: M09 retained visual - incremental processing - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - How Delta Lake Supercharges Data Lakes - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt\\_inc](#)

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Replay an older update after a newer one. A weak implementation regresses trusted state or its watermark.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to add incremental state and rerun-safe identity - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P01-07

# Build trusted PostgreSQL/dbt-style transformations

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

First create one trusted current row per order\_id, then join products on product\_id and customers on customer\_id. Assert both lookup keys are unique. Compute order revenue once at this grain. Build the paid summary from the current table, not by appending a second revenue contribution each time an order update arrives.

## Artifact and completion check

Create models/fct\_current\_orders.sql. Day 1 paid revenue is 2150.00. After day 2 it is 2940.00. Show the contribution from O1004 and the two new orders, rather than adjusting an aggregate to force the answer.

Controlled phase 7 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.

4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build trusted postgresql/dbt-style transformations - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P01-08

# Add tests and source-to-target reconciliation

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

### [2. M09 retained visual - dbt tests/key integrity](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use three independent controls: unique current keys, input disposition counts, and paid revenue calculated directly from accepted current orders. A test that compares one query to the same query inside another function proves little. Reconcile the valid order table before and after joining dimensions to catch fan-out.

## Artifact and completion check

Create tests/test\_orders\_contract.py. Keep  $12 = 9 + 1 + 2$  for day 1 and  $3 = 3 + 0 + 0$  for day 2; also keep 11 unique final keys and 2940.00 paid revenue.

Controlled phase 8 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define acceptance rules before reading outputs.
2. Separate business-invalid records from infrastructure failure.

3. Write accepted and quarantine populations durably, then reconcile counts.
4. Inject one invalid case and prove the reason is clear.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Accepted + quarantined must reconcile to the processed input population.
- Invalid reasons must be actionable; infrastructure failure must not be mislabeled as bad data.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: M09 retained visual - dbt tests/key integrity - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

[OFFICIAL/CURRENT: dbt\\_tests](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to add tests and source-to-target reconciliation - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P01-09

# Automate the repeatable run/orchestration path

## Watch first

### [1. Airflow 3 visual retained from M10](#)

Optional visual - no verified bounded range

### [2. Astronomer - Airflow 101: Essential Tips for Beginners](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Make one command accept an explicit logical batch date and source directory. It should acquire, validate, transform, check, publish and then commit progress. An Airflow task can call this command. The business code must not infer the historical input from today's date. Stop downstream publication when a critical task fails.

## Artifact and completion check

Create run\_pipeline.py. Run both supplied dates in order, then rerun only the second date. Save the exact commands and final controls; a green scheduler badge alone is not evidence.

Controlled phase 9 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the dependency graph before the scheduler code.

2. Make each step rerunnable and pass state through durable interfaces rather than hidden local memory.
3. Run the happy path, then remove one input or fail one step.
4. Recover and prove the rerun does not duplicate or skip business results.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Dependency order is explicit and rerunnable; progress is committed only after durable downstream success.
- Demonstrate one missing-input or failed-task recovery.

## Useful video / current reference

[VIDEO: Airflow 3 visual retained from M10 - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Astronomer - Airflow 101: Essential Tips for Beginners - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: airflow](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Let a downstream publish step fail after an upstream checkpoint advances. The next run skips the missing business output.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to automate the repeatable run/orchestration path - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-10

# Inject failure, diagnose, repair and prove recovery

## Watch first

### [1. Data Engineering Zoomcamp - ingestion with dlt](#)

Optional visual - no verified bounded range. Use the ingestion demonstration for acquisition and validation context; implement the project-specific quarantine, identity and recovery rules below.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use a copied source directory. First preserve a successful output. Inject changed bytes under a reused raw identity, or stop after preparing output but before publishing progress. On failure, show that the previous published output and committed progress remain the reader's truth. Remove the fault and rerun the same logical batch.

## Artifact and completion check

Create evidence/failure-and-recovery.md. Compare before, failure and recovery snapshots. A failure log without state comparisons does not prove recovery safety.

Controlled phase 10 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Trigger the supplied failure without deleting the original evidence.

2. Capture the first error/log/result and identify the smallest broken boundary.
3. Repair only that boundary and rerun from the last safe durable state.
4. Reconcile the recovered result and record what alert/runbook step would shorten future diagnosis.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Reproduce the failure, preserve evidence, identify the smallest failed boundary, repair, rerun and prove recovery.
- The recovery must not hide data loss by advancing state.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Treat an infrastructure write failure as a bad record and continue. You lose recoverable business data while dashboards appear green.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to inject failure, diagnose, repair and prove recovery - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-11

# Finish README, runbook, architecture and evidence index

## Watch first

### [1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

### [2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Write the README for a person opening the project for the first time: purpose, input grain, required environment, one setup command, one run command, expected controls and cleanup limited to disposable output. The runbook answers what to inspect when revenue differs or a duplicate arrives. Link each claim to a saved result.

## Artifact and completion check

Create README.md and docs/runbook.md. A reader can reproduce 11 current orders and 2940.00 without asking which terminal, folder or input file you used.

Controlled phase 11 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Draw source -> raw -> validation -> transform -> serving -> consumer boundaries.
2. Mark durable writes, checkpoints/state and security boundaries.

3. Record at least two alternatives and why one is rejected.
4. Save the diagram plus an ADR so reviewers can challenge your reasoning.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Every major component must trace to a requirement/constraint.
- Record at least one rejected alternative and the evidence that would make you revisit the decision.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: git](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to finish readme, runbook, architecture and evidence index - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected

control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P01-12

# Clean-clone rerun and project defense

## Watch first

### [1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

### [2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Create a new checkout and a new output directory. Install from the declared dependency file and execute only the documented commands. Do not copy your old database, state or virtual environment. Explain why the final table has fewer rows than the number of delivered source rows and why a repeated batch does not add revenue.

## Artifact and completion check

Create evidence/clean-rerun.md. Record the commit, environment versions, commands, 11/2940.00 controls, failure proof and a short defense of the update rule.

Controlled phase 12 of 50 | Project: ParcelFlow Reliable Batch Pipeline | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Operations needs daily shipped-order counts, paid revenue and hub-level exception visibility by 07:00. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Clone/copy into a clean directory with no hidden generated state.
2. Follow only the README and dependency file.

3. Run from source to final evidence, then compare expected controls.
4. Defend one trade-off, one failure/recovery and one known limitation without opening your notes.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: git](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to clean-clone rerun and project defense - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02 | SaaS Billing Analytics Engineering

A subscription SaaS company needs governed billing marts. Build dbt-style sources/staging/dimensions/facts, tests, incremental logic, history and reproducible orchestration without fan-out or hidden grain changes.

Business need: Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage.

Working folder: 17\_M18/practice/P02\_analytics\_engineering/

## P02-01

# Read the analytics-engineering brief and define outputs

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Define an invoice fact with one row per `invoice_id`. A paid-revenue mart has one row per invoice month and customer segment. The same customer can hold multiple subscriptions. Consequently, an invoice reaches its customer through `subscription_id`; joining all invoices to all of a customer's subscriptions would invent revenue.

## Artifact and completion check

Create `docs/analytics-contract.md`. Explain the route `invoice -> subscription -> customer` and state which table contributes the segment and which contributes the amount.

Controlled phase 13 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the supplied project brief once without coding.
2. Write the business questions, decision deadline, data consumers and non-functional constraints.
3. List what is evidence versus assumption.

4. Define the exact artifact that will prove the phase complete before creating code.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to read the analytics-engineering brief and define outputs - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-02

# Profile billing/customer sources and state grains

## Watch first

### [1. Python for Data Analytics - profiling with pandas](#)

Optional visual - no verified bounded range. Focus on DataFrame inspection, nulls, duplicates and grouped counts. Then profile the project files, not the instructor dataset.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Profile the four files independently: 6 customers, 3 plans, 7 subscriptions and 12 invoices. There are 10 PAID invoices. C001 has both S001 and S007, a deliberate check against joining on customer alone. Uniqueness of customer\_id says nothing about uniqueness of customer\_id in subscriptions.

## Artifact and completion check

Create evidence/source-profile.json. Save distinct-key counts and orphan checks for every relationship. Locate the customer with two subscriptions and explain why it is legitimate.

Controlled phase 14 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Copy source files into the working project without altering the originals.

2. Record row counts and the meaning of one row for every source.
3. Check keys, duplicates, missing values, type surprises and cross-source relationships.
4. Save a short data-quality note before you decide what to repair.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- State row grain, candidate keys, null/duplicate counts and at least three concrete defects.
- Do not clean the source before preserving the profiling evidence.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Pretend the first 20 rows represent the whole source. A duplicate or malformed key later in the file then breaks your assumed grain.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to profile billing/customer sources and state grains - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-03

# Design facts, dimensions, keys and serving grain

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Let `invoice_id` identify the fact. Keep customer and plan attributes in dimensions or explicitly joined fact columns, but do not change invoice grain. A dimension key identifies one current member; if you later keep history, an effective-date join must select exactly one historical version for each invoice.

## Artifact and completion check

Create `docs/model-contracts.md`. Draw the keys on both ends of each relationship and prove the fact still has 12 rows after enrichment.

Controlled phase 15 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.
4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to design facts, dimensions, keys and serving grain - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P02-04

# Create dbt project, sources and dependency graph

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Start with a working connection and source definitions, then add models one dependency at a time. `source()` names the physical ingested tables; `ref()` names a model built by dbt. These references form the dependency graph. A SQL file outside the configured model paths is not automatically part of that graph.

## Artifact and completion check

Create `dbt_project.yml` and `models/sources.yml`. Run `dbt debug`, `dbt parse` and a selected staging build using the module's installed version. Save the actual model names and dependency order.

Controlled phase 16 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.

4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to create dbt project, sources and dependency graph - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P02-05

# Build thin staging models with explicit contracts

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Staging should expose explicit columns, predictable names and valid types while retaining source identity. Convert amount to an exact decimal and invoice\_date to a date before grouping. Avoid SELECT \* because a new source column can silently change the model contract. Do not hide duplicate invoice IDs with DISTINCT until you have explained the duplicate policy.

## Artifact and completion check

Create models/staging/stg\_invoices.sql. The base staging table retains 12 invoices. A source contract failure should be visible, not converted into a successful empty model.

Controlled phase 17 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.
4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build thin staging models with explicit contracts - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P02-06

# Build dimensional marts without fan-out

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Join invoices to subscriptions on subscription\_id, then subscriptions to customers on customer\_id. Aggregate only PAID invoices for the paid mart. The six month/segment totals are August ENT 498, MID 148, SMB 148; September ENT 249, MID 49, SMB 148. Their sum is 1240.00.

## Artifact and completion check

Create models/marts/fct\_invoices.sql. The fact has 12 unique invoice IDs and the mart has 6 rows. Reconcile 10 paid invoices and 1240.00 independently from raw invoices.

Controlled phase 18 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.
4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Join a one-to-many table without controlling grain. Revenue looks larger while every SQL statement still succeeds.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build dimensional marts without fan-out - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-07

# Add data tests and critical quality gates

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

### [2. M09 retained visual - dbt tests/key integrity](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Test `invoice_id` for not-null and uniqueness and `subscription_id` for a valid relationship. Add a business reconciliation query for paid revenue. Relationship tests prevent orphan invoices; uniqueness prevents fan-out; reconciliation prevents a plausible-looking but incorrect total. A warning is not a publishing block unless your run policy makes it one.

## Artifact and completion check

Create `models/schema.yml` and `tests/revenue_control.sql`. Use the supplied duplicate and orphan files on copies. Each critical failure must prevent publishing a fresh trusted mart.

Controlled phase 19 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define acceptance rules before reading outputs.
2. Separate business-invalid records from infrastructure failure.

3. Write accepted and quarantine populations durably, then reconcile counts.
4. Inject one invalid case and prove the reason is clear.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Accepted + quarantined must reconcile to the processed input population.
- Invalid reasons must be actionable; infrastructure failure must not be mislabeled as bad data.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: M09 retained visual - dbt tests/key integrity - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

[OFFICIAL/CURRENT: dbt\\_tests](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to add data tests and critical quality gates - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-08

# Implement incremental model/update behavior

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

### [2. M09 retained visual - incremental processing](#)

Optional visual - no verified bounded range

### [3. Databricks - How Delta Lake Supercharges Data Lakes](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

The supplied invoices contain `invoice_date` but no reliable `updated_at` or CDC sequence. Therefore `invoice_date` alone cannot detect corrections to an old invoice. For this small fixture, a full rebuild is the defensible baseline. If you demonstrate incremental processing, create and label a versioned extension fixture with a documented update timestamp or source sequence; do not pretend that field exists in the original data.

## Artifact and completion check

Create `docs/incremental-policy.md`. Show why an old invoice changing OPEN to PAID would be missed by `invoice_date > watermark`. Propose an overlap/reconciliation or CDC solution and name its assumptions.

Controlled phase 20 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define ordering/version semantics and candidate progress.
2. Apply changes to a temporary/transactional result, validate it, then commit progress.
3. Replay the same batch and then an older batch.
4. Prove business state and progress remain monotonic under the stated contract.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Identical replay leaves trusted business state unchanged.
- Stale input cannot move progress backwards; equal-version ambiguity is surfaced rather than guessed.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: M09 retained visual - incremental processing - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - How Delta Lake Supercharges Data Lakes - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

[OFFICIAL/CURRENT: dbt\\_inc](#)

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Replay an older update after a newer one. A weak implementation regresses trusted state or its watermark.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to implement incremental model/update behavior - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P02-09

# Preserve history with snapshot/SCD reasoning

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

A single source snapshot cannot prove historical tracking. Make a copied second snapshot in which one customer changes segment, recording a clear observation time. Keep the original as the first snapshot. Show that the prior version closes and a new version opens, with no overlapping effective intervals. Label this as a controlled extension to the supplied data.

## Artifact and completion check

Create snapshots/customer\_history.sql and evidence/history.md. Explain whether historical invoices use the customer's old segment or the current segment. Implement the chosen business question consistently.

Controlled phase 21 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the phase title as a deliverable, not a reading assignment.
2. Write the invariant and expected control before implementation.
3. Build the smallest evidence that proves the invariant.

4. Change one input and prove the result still reconciles.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to preserve history with snapshot/scd reasoning - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-10

# Add orchestration and reproducible run order

## Watch first

### [1. Airflow 3 visual retained from M10](#)

Optional visual - no verified bounded range

### [2. Astronomer - Airflow 101: Essential Tips for Beginners](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Run source ingestion and source checks before dbt staging, then dimensions/facts, then business controls and publication. Keep the selection explicit so a partial run does not accidentally skip a critical ancestor. For this project, serial execution is acceptable; dependencies and truthful failure behavior matter more than parallelism.

## Artifact and completion check

Create `run_analytics.sh` or an orchestration DAG. Introduce a bad source relationship and show the serving publication does not proceed. Fix the input and rerun from the documented entry point.

Controlled phase 22 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the dependency graph before the scheduler code.

2. Make each step rerunnable and pass state through durable interfaces rather than hidden local memory.
3. Run the happy path, then remove one input or fail one step.
4. Recover and prove the rerun does not duplicate or skip business results.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Dependency order is explicit and rerunnable; progress is committed only after durable downstream success.
- Demonstrate one missing-input or failed-task recovery.

## Useful video / current reference

[VIDEO: Airflow 3 visual retained from M10 - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Astronomer - Airflow 101: Essential Tips for Beginners - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: airflow](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Let a downstream publish step fail after an upstream checkpoint advances. The next run skips the missing business output.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to add orchestration and reproducible run order - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-11

# Inject duplicate/relationship/partial-run failures

## Watch first

### [1. M09 retained visual - dbt tests/key integrity](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

The duplicate file contains a repeated invoice identity; the orphan file refers to S999. Test these separately so the first failure does not hide the second. Also stop a build between staging and mart publication. Readers should either see the prior validated mart or a clearly unavailable result, not a half-built mixed version.

## Artifact and completion check

Create evidence/negative-cases.md. Record the failing key, expected test, actual failure and previous published controls. After recovery, return to 12 facts, 6 mart rows and 1240.00.

Controlled phase 23 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Trigger the supplied failure without deleting the original evidence.
2. Capture the first error/log/result and identify the smallest broken boundary.

3. Repair only that boundary and rerun from the last safe durable state.
4. Reconcile the recovered result and record what alert/runbook step would shorten future diagnosis.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Reproduce the failure, preserve evidence, identify the smallest failed boundary, repair, rerun and prove recovery.
- The recovery must not hide data loss by advancing state.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt tests/key integrity - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt\\_tests](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Treat an infrastructure write failure as a bad record and continue. You lose recoverable business data while dashboards appear green.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to inject duplicate/relationship/partial-run failures - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-12

# Document lineage, runbook, limitations and evidence

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Connect a single paid invoice from raw file to staging to fact to month/segment total. Document exact decimals, tie/conflict policies, incremental limitations and the generated-history demonstration. A credible portfolio describes where its guarantees stop. Do not claim source CDC or historical truth that the supplied snapshot cannot establish.

## Artifact and completion check

Create docs/lineage-and-limitations.md. Provide one lineage example with real IDs and a runbook entry for revenue changing unexpectedly after a source correction.

Controlled phase 24 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the phase title as a deliverable, not a reading assignment.
2. Write the invariant and expected control before implementation.
3. Build the smallest evidence that proves the invariant.
4. Change one input and prove the result still reconciles.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to document lineage, runbook, limitations and evidence - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P02-13

# Clean-clone build and analytics-platform defense

## Watch first

### [1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

### [2. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Build from a new checkout and empty target schema using documented source files. Execute tests and save the resulting mart, not only a successful dbt process exit. Defend the invoice grain and explain the intentionally dangerous many-subscription customer. Explain the full-refresh choice before proposing an incremental alternative.

## Artifact and completion check

Create evidence/clean-build.md. Reproduce all six mart rows and the 1240.00 control; keep both failure cases and their successful recoveries.

Controlled phase 25 of 50 | Project: SaaS Billing Analytics Engineering | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Finance needs monthly recurring revenue, invoice collections and plan/customer history with explainable lineage. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Clone/copy into a clean directory with no hidden generated state.
2. Follow only the README and dependency file.

3. Run from source to final evidence, then compare expected controls.
4. Defend one trade-off, one failure/recovery and one known limitation without opening your notes.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: git](#)

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to clean-clone build and analytics-platform defense - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03 | ClickStream Scalable Lakehouse

A commerce site produces clickstream events with campaign/product reference data. Build a Spark/lakehouse design with explicit schemas, join controls, partition reasoning, Delta-style idempotency, medallion contracts and schema-evolution recovery.

Business need: Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay.

Working folder: 17\_M18/practice/P03\_scalable\_lakehouse/

## P03-01

# Read the scalable-lakehouse brief and define scale risks

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [3. Databricks - Medallion Architecture: Bronze, Silver, Gold](#)

Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

The fixture has 60 events; it is designed for correctness, not proof of big-data performance. Name the future scale risk you want to study: shuffle on a large join, a hot key, excessive partitions or tiny output files. Start with one Spark worker on the local laptop. Distributed APIs do not make every small workload faster.

## Artifact and completion check

Create docs/scale-hypotheses.md. Separate measured results on 60 rows from predictions about millions of rows. Never report a fabricated production throughput.

Controlled phase 26 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the supplied project brief once without coding.
2. Write the business questions, decision deadline, data consumers and non-functional constraints.
3. List what is evidence versus assumption.
4. Define the exact artifact that will prove the phase complete before creating code.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - Medallion Architecture: Bronze, Silver, Gold - Open video](#)

Watch: Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to read the scalable-lakehouse brief and define scale risks - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-02

# Profile event schema, grain, keys and timestamps

## Watch first

### [1. Spark transformations and data processing](#)

Optional visual - no verified bounded range. Focus on explicit schemas, joins and transformations, then apply the supplied event contract. Course failure fixtures determine correctness.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

One row is one event\_id. Profile 60 unique IDs over three dates, with 20 purchase events. product\_id and campaign\_id connect to five products and four campaigns. Parse event\_time as UTC and validate amount before aggregation. A view event and a purchase event are different business actions even if they belong to one user.

## Artifact and completion check

Create evidence/event-profile.json. Count by event\_type and date, check both foreign keys, and independently calculate purchase amount 885.

Controlled phase 27 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Copy source files into the working project without altering the originals.

2. Record row counts and the meaning of one row for every source.
3. Check keys, duplicates, missing values, type surprises and cross-source relationships.
4. Save a short data-quality note before you decide what to repair.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- State row grain, candidate keys, null/duplicate counts and at least three concrete defects.
- Do not clean the source before preserving the profiling evidence.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Pretend the first 20 rows represent the whole source. A duplicate or malformed key later in the file then breaks your assumed grain.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to profile event schema, grain, keys and timestamps - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-03

# Create Spark environment and explicit schemas

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Create the Spark session and explicit raw schema in a small reusable module. Preserve amount and timestamps as text until validation can distinguish malformed values. Use tolerant conversion such as `try_cast` and `try_to_timestamp`; check the resulting nulls and format before publishing. Record Spark and Java versions so a reviewer can reproduce API behavior.

## Artifact and completion check

Create `src/session_and_schema.py`. The valid file loads 60 events. The malformed timestamp/NaN failure file produces a rejected record with its raw payload and reason, not a silent null in Gold.

Controlled phase 28 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.

2. Inspect partitions/explain plan and predict the bottleneck before optimizing.
3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to create spark environment and explicit schemas - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-04

# Build deterministic distributed transformations

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Express the result as column operations: parse, validate, select, derive event\_date, join and aggregate. Avoid collecting the whole input into a Python list to perform the actual pipeline. Treat duplicate event IDs with changed payload as conflicts unless a version contract exists; the event file does not provide an update version.

## Artifact and completion check

Create src/transform\_events.py. Repartition the valid input and rerun. Sorted business output must stay identical even though physical row order or file names may change.

Controlled phase 29 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.
2. Inspect partitions/explain plan and predict the bottleneck before optimizing.

3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build deterministic distributed transformations - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-05

# Join reference data and validate cardinality

## Watch first

### [1. Spark transformations and data processing](#)

Optional visual - no verified bounded range. Focus on explicit schemas, joins and transformations, then apply the supplied event contract. Course failure fixtures determine correctness.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Join products and campaigns only after proving their keys are unique. Broadcast can be a sensible physical plan for these tiny reference tables, but it does not cure duplicate dimension keys. Count rows before and after both joins and inspect unmatched references before deciding between rejection and an explicit unknown member.

## Artifact and completion check

Create evidence/join-controls.json. The valid enriched event table still contains 60 rows and 60 distinct event IDs. Purchase amount remains 885.

Controlled phase 30 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the phase title as a deliverable, not a reading assignment.

2. Write the invariant and expected control before implementation.
3. Build the smallest evidence that proves the invariant.
4. Change one input and prove the result still reconciles.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Join a one-to-many table without controlling grain. Revenue looks larger while every SQL statement still succeeds.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to join reference data and validate cardinality - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-06

# Reason about partitions, shuffle and skew

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use the supplied skew fixture in a separate experiment. Group by `user_id` and show the largest group before changing partitions. More partitions cannot split one key's aggregation work unless the algorithm changes. If you salt a hot key, merge partial aggregates back to the original grain and prove totals remain correct.

## Artifact and completion check

Create evidence/skew-experiment.md. Record the actual key-frequency distribution, partition choice and equivalent final totals. Treat timings as local observations, not a benchmark claim.

Controlled phase 31 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.
2. Inspect partitions/explain plan and predict the bottleneck before optimizing.

3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Partition on a high-cardinality ID or let one hot key dominate a shuffle. File count or one straggler becomes the bottleneck.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to reason about partitions, shuffle and skew - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-07

# Write Parquet with justified partition strategy

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

For the 60-row example, an unpartitioned output may be simpler. If you demonstrate partitioning by event\_date, expect three dates and explain how a date filter prunes work. Do not partition by event\_id: that creates tiny directories with almost no useful grouping. File count and table row count are different measurements.

## Artifact and completion check

Create output/silver\_events/ and docs/partition-choice.md. Read your written Parquet back, validate 60 events and 885 purchase amount, and compare its schema with your declared contract.

Controlled phase 32 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.

2. Inspect partitions/explain plan and predict the bottleneck before optimizing.
3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

[OFFICIAL/CURRENT: parquet](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to write parquet with justified partition strategy - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control

first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-08

# Apply Delta-style incremental merge/idempotency

## Watch first

### [1. M09 retained visual - incremental processing](#)

Optional visual - no verified bounded range

### [2. Databricks - How Delta Lake Supercharges Data Lakes](#)

Optional visual - no verified bounded range

### [3. Databricks - Medallion Architecture: Bronze, Silver, Gold](#)

Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use event\_id as immutable event identity. Exact repeats are no-ops; changed payload under the same identity is rejected because there is no supplied event version. A Delta MERGE can insert only unmatched IDs after conflict checks. Do not increment amount on every match, which would multiply revenue on replay.

## Artifact and completion check

Create src/merge\_events.py. Apply the same 60 events twice. The table stays at 60 events with purchase amount 885. Show the conflict failure on a copied changed event.

Controlled phase 33 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define ordering/version semantics and candidate progress.
2. Apply changes to a temporary/transactional result, validate it, then commit progress.
3. Replay the same batch and then an older batch.
4. Prove business state and progress remain monotonic under the stated contract.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Identical replay leaves trusted business state unchanged.
- Stale input cannot move progress backwards; equal-version ambiguity is surfaced rather than guessed.

## Useful video / current reference

[VIDEO: M09 retained visual - incremental processing - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - How Delta Lake Supercharges Data Lakes - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - Medallion Architecture: Bronze, Silver, Gold - Open video](#)

Watch: Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

[OFFICIAL/CURRENT: dbt\\_inc](#)

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Replay an older update after a newer one. A weak implementation regresses trusted state or its watermark.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to apply delta-style incremental merge/idempotency - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control

first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

P03-09

# Define Bronze/Silver/Gold lakehouse contracts

## Watch first

### [1. Databricks - Medallion Architecture: Bronze, Silver, Gold](#)

Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

### [2. Databricks - How Delta Lake Supercharges Data Lakes](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Bronze preserves acquisitions and source metadata. Silver holds one valid, enriched event per event\_id. Gold has one row per event\_date and marketing channel with purchase\_amount. A bronze row count can exceed Silver after replay or rejection; this is acceptable only when the disposition is explained.

## Artifact and completion check

Create docs/layer-contracts.md. Gold has 12 date/channel rows, daily totals 300, 270 and 315, and overall total 885. Explain each layer's grain.

Controlled phase 34 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define each layer as a contract, not just a folder name.

2. Bronze preserves source evidence; Silver applies validated business structure; Gold serves declared consumer grain.
3. Write promotion checks between layers.
4. Reject any promotion that cannot reconcile to its upstream population.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Databricks - Medallion Architecture: Bronze, Silver, Gold - Open video](#)

Watch: Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

[VIDEO: Databricks - How Delta Lake Supercharges Data Lakes - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to define bronze/silver/gold lakehouse contracts - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-10

# Handle schema evolution and supplied failure cases

## Watch first

### [1. Spark transformations and data processing](#)

Optional visual - no verified bounded range. Focus on explicit schemas, joins and transformations, then apply the supplied event contract. Course failure fixtures determine correctness.

### [2. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Treat `device_type` in the additive fixture as a declared optional extension. Preserve it in raw, then explicitly decide whether to add a nullable Silver column or ignore it for this version. It repeats E0001, so comparing payloads must first use the declared business schema. A new optional field is different from a changed amount under an existing event ID.

## Artifact and completion check

Create evidence/schema-evolution.md. The additive replay adds no business event. The bad timestamp/NaN record is quarantined. Keep separate evidence for each policy.

Controlled phase 35 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Trigger the supplied failure without deleting the original evidence.

2. Capture the first error/log/result and identify the smallest broken boundary.
3. Repair only that boundary and rerun from the last safe durable state.
4. Reconcile the recovered result and record what alert/runbook step would shorten future diagnosis.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Reproduce the failure, preserve evidence, identify the smallest failed boundary, repair, rerun and prove recovery.
- The recovery must not hide data loss by advancing state.

## Useful video / current reference

[VIDEO: freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\] - Open video](#)

Watch: 01:04-05:41 - bounded Git mental-model visual; project commands are in the course labs

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Treat an infrastructure write failure as a bad record and continue. You lose recoverable business data while dashboards appear green.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to handle schema evolution and supplied failure cases - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-11

# Inspect plans/performance and prove equivalent results

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Capture explain output for the same logical transformation before and after one deliberate change, such as broadcasting the small references. Look for exchanges and join type. Compare complete sorted business results, not merely row count. A plan change is evidence about execution, while a speed claim requires repeatable measurement.

## Artifact and completion check

Create evidence/physical-plans.md. Show equal keys and amounts for both plans, the relevant plan difference and measured local timings with machine limitations.

Controlled phase 36 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.
2. Inspect partitions/explain plan and predict the bottleneck before optimizing.

3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to inspect plans/performance and prove equivalent results - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-12

# Map the design to Fabric/cloud deployment

## Watch first

### [1. Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026](#)

04:49-27:35 - pipeline overview and Copy Data activity

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Map raw files to OneLake Files, trusted tables to Delta tables, compute to a notebook and the scheduled run to a pipeline. State the workspace identity and who may read each layer. Keep the local route executable when tenant access is unavailable; a mapping diagram is design evidence, not a completed cloud deployment.

## Artifact and completion check

Create docs/fabric-deployment.md. If access is available, save an actual notebook/pipeline run and reconciliation. Otherwise label the live step pending and retain the local result honestly.

Controlled phase 37 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Map project artifacts to Fabric/OneLake/Lakehouse/Notebook/Pipeline/SQL-endpoint items.
2. List identities, least-privilege permissions and secret locations.
3. Define dev/test/prod promotion and cleanup.

4. If you have tenant access, capture real run evidence; otherwise keep the design and mark live execution pending.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Map local project artifacts to explicit Fabric items/identities/secret boundaries.
- If tenant access is unavailable, label live execution pending instead of inventing portal evidence.

## Useful video / current reference

[VIDEO: Aleksii Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Open video](#)

Watch: 04:49-27:35 - pipeline overview and Copy Data activity

[OFFICIAL/CURRENT: fabric](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to map the design to fabric/cloud deployment - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## P03-13

# Clean rerun, reconcile outputs and defend trade-offs

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Rebuild into fresh output and checkpoint directories. Run the valid data, exact replay and separate bad/additive/skew cases using the documented policy. Defend why event\_id is the key, why physical file names may differ and why correctness checks do not prove large-scale performance.

## Artifact and completion check

Create evidence/clean-rerun.md. Save 60 valid events, 20 purchases, 12 Gold rows and 885 total, plus the specific failure and evolution evidence.

Controlled phase 38 of 50 | Project: ClickStream Scalable Lakehouse | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Growth and product teams need trustworthy event funnels while engineering must handle volume, skew, late/schema-changed events and replay. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Clone/copy into a clean directory with no hidden generated state.
2. Follow only the README and dependency file.
3. Run from source to final evidence, then compare expected controls.

4. Defend one trade-off, one failure/recovery and one known limitation without opening your notes.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to clean rerun, reconcile outputs and defend trade-offs - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP | OmniRetail Production Data Platform Capstone

Combine orders, payments, customer data and inventory changes into one production-grade design. Integrate ingestion, quality, dbt/warehouse serving, orchestration, scale-aware Spark/Delta choices, Fabric deployment planning, CI/CD, observability and incident recovery.

Business need: Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability.

Working folder: 17\_M18/practice/CAP\_production\_capstone/

## CAP-01

# Read capstone requirements and write source contracts

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

This capstone combines customers, orders, payments and inventory change events. State a separate grain for each. A payment\_id identifies a payment event; it does not by itself prevent two different capture events from charging one order twice. inventory source\_seq establishes source ordering in this fixture, while delta\_qty is an additive change.

## Artifact and completion check

Create docs/source-contracts.md. Profile 8 customers, 12 orders, 12 payment events and 8 inventory changes. State allowed statuses, foreign keys, time rules and duplicate policies.

Controlled phase 39 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Read the supplied project brief once without coding.
2. Write the business questions, decision deadline, data consumers and non-functional constraints.
3. List what is evidence versus assumption.

4. Define the exact artifact that will prove the phase complete before creating code.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to read capstone requirements and write source contracts - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-02

# Design architecture and record major ADRs

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Choose a batch schedule unless a business need requires lower latency. Separate raw acquisition, trusted business tables and serving publication. An architecture decision record contains the requirement, alternatives, choice and consequence. For example, publish a validated generation together rather than letting readers combine yesterday's orders with today's payments.

## Artifact and completion check

Create docs/ADRs/001-serving-and-recovery.md. Document one concrete failure the design tolerates and one it does not. Name the recovery artifact for each stateful component.

Controlled phase 40 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Draw source -> raw -> validation -> transform -> serving -> consumer boundaries.
2. Mark durable writes, checkpoints/state and security boundaries.
3. Record at least two alternatives and why one is rejected.

4. Save the diagram plus an ADR so reviewers can challenge your reasoning.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Every major component must trace to a requirement/constraint.
- Record at least one rejected alternative and the evidence that would make you revisit the decision.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to design architecture and record major adrs - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-03

# Build multi-source ingestion and raw evidence

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Acquire every required source with identity and byte count before transformation. A missing orders input must fail this logical run; it must not be interpreted as zero sales. Preserve source provenance for payments and inventory even when the serving table combines them. Keep customer email out of logs and general serving outputs.

## Artifact and completion check

Create `src/ingest_all.py`. Remove `orders.csv` in a copied source directory and prove no new trusted generation or progress is published.

Controlled phase 41 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Create an immutable landing path using source + run identity.
2. Write bytes first, calculate hash/size/metadata, then append manifest state.
3. Test identical replay and changed-content collision.
4. Prove the first landed bytes remain retrievable.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Raw bytes must be immutable and retrievable by run/source identity.
- A repeated run must not silently overwrite earlier evidence.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Reuse a run ID with changed bytes. If the earlier raw object disappears, your audit trail is not immutable.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build multi-source ingestion and raw evidence - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-04

# Implement critical quality, quarantine and reconciliation

## Watch first

### [1. M09 retained visual - dbt tests/key integrity](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Validate unique order keys and event IDs, relationships and exact money. Define at most one CAPTURED payment per order for this supplied business case; a second capture with another payment\_id needs review. Do not sum repeated capture events and call the resulting overcharge new revenue. Inventory must also reject a reused global source sequence.

## Artifact and completion check

Create tests/test\_business\_contracts.py. The duplicate-payment and stale-inventory incident packs fail the corresponding control before publication. Keep the conflicting IDs and reasons.

Controlled phase 42 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Define acceptance rules before reading outputs.
2. Separate business-invalid records from infrastructure failure.
3. Write accepted and quarantine populations durably, then reconcile counts.
4. Inject one invalid case and prove the reason is clear.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Accepted + quarantined must reconcile to the processed input population.
- Invalid reasons must be actionable; infrastructure failure must not be mislabeled as bad data.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt tests/key integrity - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt\\_tests](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to implement critical quality, quarantine and reconciliation - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-05

# Build trusted warehouse/dbt serving layer

## Watch first

### [1. M09 retained visual - dbt model/project structure](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Aggregate valid captures by order first, then LEFT JOIN the one-row-per-order result to orders. Keep orders with no captured payment. The order total is 1380.00 and captured total is 1240.00. OR009 has a declined payment and contributes the 140.00 gap. This difference is explained business state, not missing data to invent.

## Artifact and completion check

Create models/fct\_order\_payments.sql. Return 12 order facts, 11 captured payments and OR009 as the unpaid order. Explain why an INNER JOIN to captured payments would hide the gap.

Controlled phase 43 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and expected\_controls.json. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the model grain above the SQL.
2. Keep staging thin and deterministic; make keys and relationships explicit.
3. Build marts from controlled grains and test fan-out before accepting aggregates.
4. Save row counts/key checks plus one business reconciliation query.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Model grain is written before SQL; row-count/fan-out checks prove the model did not silently duplicate facts.
- Sources, refs/dependencies, tests and materialization choices are documented.

## Useful video / current reference

[VIDEO: M09 retained visual - dbt model/project structure - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: dbt](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to build trusted warehouse/dbt serving layer - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-06

# Orchestrate dependencies, retries and bounded backfills

## Watch first

### [1. Airflow 3 visual retained from M10](#)

Optional visual - no verified bounded range

### [2. Astronomer - Airflow 101: Essential Tips for Beginners](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use explicit logical intervals and a bounded set of available fixture dates. Build dependencies around data readiness, not arbitrary delays. Retry transient acquisition problems with limits; a contract conflict needs diagnosis and should not loop forever. The same historical run must converge to the same business output.

## Artifact and completion check

Create dags/capstone.py. Backfill a supplied interval, inject one failure, then retry that interval. Compare committed progress and business totals across all three states.

Controlled phase 44 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Write the dependency graph before the scheduler code.

2. Make each step rerunnable and pass state through durable interfaces rather than hidden local memory.
3. Run the happy path, then remove one input or fail one step.
4. Recover and prove the rerun does not duplicate or skip business results.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Dependency order is explicit and rerunnable; progress is committed only after durable downstream success.
- Demonstrate one missing-input or failed-task recovery.

## Useful video / current reference

[VIDEO: Airflow 3 visual retained from M10 - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Astronomer - Airflow 101: Essential Tips for Beginners - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: airflow](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to orchestrate dependencies, retries and bounded backfills - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-07

# Use Spark/Delta where scale justifies it

## Watch first

### [1. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [2. DataTalksClub - Apache Spark / PySpark](#)

Optional visual - no verified bounded range

### [3. Databricks - Medallion Architecture: Bronze, Silver, Gold](#)

Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Use Spark or Delta for a named purpose such as scalable event transformations or table-level merge semantics, while retaining simpler SQL where it is sufficient. A Delta commit is atomic for one table; multiple independently committed tables are not automatically one business transaction. Explain how readers see a consistent published generation.

## Artifact and completion check

Create docs/compute-choice.md. Reconcile the Spark/Delta result against the small business reference. Record what was actually executed and which scale assumptions remain theoretical.

Controlled phase 45 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Use an explicit schema and establish a correctness baseline first.
2. Inspect partitions/explain plan and predict the bottleneck before optimizing.
3. Change partition/join/layout strategy one variable at a time.
4. Prove result equivalence after the change and save the plan/metrics evidence.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Save explain/partition evidence and prove the optimized version returns equivalent business results.
- Performance claims must be tied to measured plan/partition evidence, not a slogan.

## Useful video / current reference

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: DataTalksClub - Apache Spark / PySpark - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Databricks - Medallion Architecture: Bronze, Silver, Gold - Open video](#)

Watch: Optional visual with publisher chapter markers; use matching Bronze/Silver/Gold chapter

[OFFICIAL/CURRENT: spark](#)

[OFFICIAL/CURRENT: sparkperf](#)

[OFFICIAL/CURRENT: delta](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to use spark/delta where scale justifies it - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-08

# Design Fabric deployment, access and secret boundaries

## Watch first

### [1. Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026](#)

04:49-27:35 - pipeline overview and Copy Data activity

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

List raw, trusted and serving locations with the identity allowed to read or write each. General analysts do not need customer email or source credentials. Keep secrets in the platform connection/secret mechanism. Use an eligible workspace when available and a truthful deployment design when it is not; do not buy capacity simply to claim completion.

## Artifact and completion check

Create docs/cloud-access-matrix.md. Provide either actual Fabric run evidence or an explicitly pending live deployment, together with a reviewed access matrix and local controls.

Controlled phase 46 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Map project artifacts to Fabric/OneLake/Lakehouse/Notebook/Pipeline/SQL-endpoint items.
2. List identities, least-privilege permissions and secret locations.
3. Define dev/test/prod promotion and cleanup.

4. If you have tenant access, capture real run evidence; otherwise keep the design and mark live execution pending.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Map local project artifacts to explicit Fabric items/identities/secret boundaries.
- If tenant access is unavailable, label live execution pending instead of inventing portal evidence.

## Useful video / current reference

[VIDEO: Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Open video](#)

Watch: 04:49-27:35 - pipeline overview and Copy Data activity

[OFFICIAL/CURRENT: fabric](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to design fabric deployment, access and secret boundaries - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-09

# Add CI/CD, monitoring, alerts and runbook

## Watch first

### [1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

### [2. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

### [3. Monte Carlo - Data Observability](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

CI should run fast contract and unit checks on supplied fixtures before deployment. Monitoring should identify a logical run and show input counts, rejects, output controls, duration and committed progress. Alert on a failed critical run or stale trusted data with a clear owner/action, rather than every harmless duplicate.

## Artifact and completion check

Create ci workflow and docs/runbook.md. Make a deliberately failing test block the build. Demonstrate one useful incident message and the runbook steps a responder follows.

Controlled phase 47 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Choose checks that must block merge/deploy versus checks that only warn.
2. Add pipeline health, freshness, volume and quality signals with ownership.
3. Write one alert -> runbook action -> escalation path.
4. Break a control locally and prove the gate/alert catches it.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets

[VIDEO: Monte Carlo - Data Observability - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: gha](#)

[OFFICIAL/CURRENT: ote](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Deploy a change with no quality gate or alert. The first evidence of failure is a stakeholder complaint.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to add ci/cd, monitoring, alerts and runbook - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-10

# Open incident pack, diagnose and recover safely

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Work on copied inputs. Test missing orders, duplicate capture and stale inventory separately. State the symptom, affected output, suspect source, decisive evidence and repair. Never erase the failure evidence by replacing it with a successful screenshot. A safe recovery uses preserved input and replay controls instead of editing a final total by hand.

## Artifact and completion check

Create evidence/incident-timeline.md. For each incident, keep last good state, rejected attempt and recovered state. All recovered business controls must match the baseline.

Controlled phase 48 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Trigger the supplied failure without deleting the original evidence.
2. Capture the first error/log/result and identify the smallest broken boundary.
3. Repair only that boundary and rerun from the last safe durable state.

4. Reconcile the recovered result and record what alert/runbook step would shorten future diagnosis.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Reproduce the failure, preserve evidence, identify the smallest failed boundary, repair, rerun and prove recovery.
- The recovery must not hide data loss by advancing state.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Treat an infrastructure write failure as a bad record and continue. You lose recoverable business data while dashboards appear green.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to open incident pack, diagnose and recover safely - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-11

# Reproduce from clean state and freeze evidence bundle

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Run from a clean checkout and fresh targets. Record the commit, dependency versions, fixture hashes and commands. Freeze a compact evidence bundle containing controls, failure/recovery proofs and the final data contract. Remove credentials and unnecessary customer-level personal data before publishing your portfolio.

## Artifact and completion check

Create evidence/RELEASE.md. Reproduce 12 order facts, 1380.00 order amount, 1240.00 captured amount and unpaid OR009 without relying on old local state.

Controlled phase 49 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Clone/copy into a clean directory with no hidden generated state.
2. Follow only the README and dependency file.
3. Run from source to final evidence, then compare expected controls.

4. Defend one trade-off, one failure/recovery and one known limitation without opening your notes.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to reproduce from clean state and freeze evidence bundle - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.

## CAP-12

# Final technical defense and Gate 7 readiness

## Watch first

### [1. Microsoft Mechanics - Azure Well-Architected Framework](#)

00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

## Work this phase with the supplied data

Explain the business problem first, then one row's journey, keys/grains, a real failure and the evidence that recovery is safe. State that inventory change totals are P01 -5, P02 -7, P03 -9, P04 -11 and P05 -4; these are net changes, not stock balances because opening stock was not supplied. Discuss what you would add for real production traffic.

## Artifact and completion check

Create docs/technical-defense.md. Answer follow-ups about duplicate captures, source corrections, partial publication, access and cost. Complete Gate A, Review A, fresh Gate B and Review B in that order.

Controlled phase 50 of 50 | Project: OmniRetail Production Data Platform Capstone | Estimated focused build time: 1.5-3.5 hours

## Why this matters

This phase converts one part of the project from an idea into reviewable engineering evidence. Executives and operations need same-day revenue/inventory truth while engineering must preserve replay, security boundaries and recoverability. The phase is not complete because a script ran once; it is complete when another engineer can see the intended grain/state boundary, reproduce the result and understand how failure is handled.

## Before you build

Open the project README and `expected_controls.json`. Work only in your project copy. Preserve supplied raw/failure fixtures unchanged. Write one sentence naming the invariant for this phase - for example 'accepted + quarantined = processed input' or 'replay does not change trusted business state' - before changing code.

## Follow with me once

1. Clone/copy into a clean directory with no hidden generated state.
2. Follow only the README and dependency file.
3. Run from source to final evidence, then compare expected controls.

4. Defend one trade-off, one failure/recovery and one known limitation without opening your notes.

## What must exist when you stop

- Save the genuine first attempt before opening any protected review.
- Keep commands, outputs and at least one validation artifact in the project evidence folder.
- Define the deliverable, expected control and completion evidence before implementation.
- A changed-input retry must still satisfy the invariant.

## Useful video / current reference

[VIDEO: Microsoft Mechanics - Azure Well-Architected Framework - Open video](#)

Watch: 00:34-01:57 categories; 01:57-04:32 recommendations; 04:32-07:22 workload recommendations

[OFFICIAL/CURRENT: arch](#)

Use rule: Watch only to visualize the interface/concept. Close the source before the independent build. A tutorial-shaped output without your own evidence, failure reasoning and changed retry does not pass.

## Break it deliberately

Change one important assumption - volume, latency, key uniqueness, file availability or permission - and observe whether the design still proves its invariant.

## Recovery ladder

Save the first failure. Confirm the input population and last safe durable state. Narrow the boundary: source/contract, transformation, sink, state/checkpoint, permissions or orchestration. Repair the smallest boundary first. Rerun from the last safe point, then reconcile the business result. Never erase the failed attempt from the evidence bundle.

## Changed retry

Change one controlled input relevant to final technical defense and gate 7 readiness - for example a date, key distribution, update version, missing file, duplicate, schema field, volume or permission. Predict the expected control first, rerun, and save proof that your invariant still holds or that the system fails safely.

## Defend it in two minutes

Explain: (1) what one row/event represents here, (2) what state is durable, (3) the failure you are protecting against, (4) the validation that proves correctness, and (5) one limitation you would address before production scale. If you cannot answer without reading code line-by-line, the phase is not mastered yet.